SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – DEBUGGING & OPTIMIZATION TASK

Course Code:	CSA0305	**Course Title:**	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	**Assessment:**	Assessment Tool 2 – Debugging & Optimisation Task
Weightage:	35% of CIA	**Total Marks:**	100
CO2 Statement:	*Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.*		
Student Name:	Shahana	**Reg. No.:**	192524479
Section / Batch:	Slot A	**Date:**	24/07/2026

Code of Conduct

I Shahana (192524479)_____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Shahana_____

Instructions

- This test contains 20 Debugging & Optimisation Task questions. Total: 100 Marks (20*5).
- When a student answer using Scenario-Based, in evaluation the answer should be with following five requirements: Identify the problem type and constraints, Select appropriate data structure, Design the Solution, Optimization and efficiency, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Search Data Structure, Stack	Binary Search, Stack Operations	1 - 5	25
Linked List Data Structure	List Operations	6 – 10	25
Queue Data Structures	Queue Operations	11 - 15	25
Queue Data Structures, Stack Notations	Queue Operations, Expression Evaluation	16 - 20	25
	GRAND TOTAL:	**100 Marks**	

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Debugging Accuracy	20	All errors corrected; code runs flawlessly	Most errors corrected; minor issues remain	Partial correction; code runs with issues	Code fails or major errors persist
Error Identification	30	Accurately identifies all logical, syntax, and edge-case errors	Identifies most major errors	Identifies some errors but misses key issues	Struggles to identify errors
Code Quality & Readability	20	Clean, modular, well-documented, follows best practices	Mostly clean and readable	Some structure but inconsistent	Poorly written, hard to understand
Explanation & Justification	20	Clear, logical, and well-justified reasoning with complexity analysis	Good explanation with some justification	Limited explanation; lacks depth	No or incorrect explanation
Testing & Validation	10	Comprehensive test cases including edge cases	Adequate test cases	Minimal testing	No testing provided

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

```c
#include <stdio.h>
int binarySearch (int arr[],int n,int key)
{
 int low=0,high=n-1;
    while(low<=high){
        int mid=low+(high-low)/2;
        if(arr[mid]==key)
            return mid;
        else if(arr[mid]<key)
            low=mid+1;
        else
            high=mid-1;
    }
    return -1;
}
int main(){
 int arr[]={2,4,6,8,10,12,14,16,18,20};
    int n=sizeof(arr)/sizeof(arr[0]);
    int key;
    printf("Array:");
    for(int i=0;i<n;i++)
    {
        printf("%d",arr[i]);
        printf("\n");
    }
    printf("Enter key to search:");
    scanf("%d",key);
    int result=binarySearch(arr,n,key);
    if (result!=-1)
        printf("Element found at index %d\n",result);
    else
        printf("Element not found");
    return 0;
}
```

stdin input (optional)

Output

```
Array:2
4
6
8
10
12
```

Run | Save | Submit Fix

```c
#include <stdio.h>
#define MAX 5
int stack[MAX];
int top=-1;
void push(int value){
    if (top==MAX-1){
        printf("Overflow\n");
        return;
    }
    stack[++top]=value;
}
int pop(){
    if(top==-1){
        printf("Underflow\n");
        return -1;
    }
    return stack[top--];
}
int main()
{
    push(10);
    push(20);
    push(30);
```

stdin input (optional)

Output

```
Popped:30
Popped:20
Popped:10
Underflow
Popped:-1
```

```c
int pop(){
    if(top==-1){
        printf("Underflow\n");
        return -1;
    }
    return stack[top--];
}
int main()
{
    push(10);
    push(20);
    push(30);
    printf("Popped:%d\n",pop());
    printf("Popped:%d\n",pop());
    printf("Popped:%d\n",pop());
    printf("Popped:%d\n",pop());
    return 0;
}
```

```c
#include <stdio.h>
#define MAX 5
int stack[MAX];
int top;
void init(){
top=-1;
}
void push(int value){
    if (top==MAX-1){
        printf("Overflow\n");
        return;
    }
    stack[++top]==value;
}
int pop(){
    if (top==-1){
        printf("Underflow\n");
        return -1;
    }
    return stack[top--];
}
int main(){
```

stdin input (optional)

Output

```
Popped:0
popped:0
popped:0
Underflow
popped:-1
```

```c
        return -1;
    }
    return stack[top--];
}
int main(){
    init();
    push(10);
    push(20);
    push(30);
    printf("Popped:%d\n",pop());
    printf("popped:%d\n",pop());
    printf("popped:%d\n",pop());
    printf("popped:%d\n",pop());
    return 0;
}
```

```c
#include <stdio.h>
#define MAX 20
char stack[MAX];
int top=-1;
void push(char c){
    stack[++top]=c;
}
char pop(){
return stack[top--];
}
int main()
{
    char postfix[MAX];
    int j=0;
    push('(');
    push('A');
    push('+');
    push('B');
    char ch=')';
    if (ch==')'){
        while (stack[top]!='('){
            postfix[j++]=pop();
```

stdin input (optional)

Output

```
Postfix so far:(A+B
,postfix
```

```c
    push('A');
    push('+');
    push('B');
    char ch=')';
    if (ch==')'){
        while (stack[top]!='('){
            postfix[j++]=pop();
        }
        pop();
    }
    postfix[j]='\0';
    printf("Postfix so far:%s\n,postfix");
          return 0;
          }
```

```c
#include <stdio.h>
#define MAX 50
char stack[MAX];
int top=-1;
void push(char c){
    stack[++top]=c;
}
char pop(){
    return stack[top--];
}
int isBalanced(char expr[]){
    for(int i=0;expr[i]!='\0';i++){
        char ch=expr[i];
        if(ch=='('||ch=='['||ch=='{'){
            push(ch);
    }
    else if(ch==')'||ch==']'||ch=='}'){
        if (top==-1) return 0;
        if((ch==')'&&stack[top]=='('||(ch==']'&&stack[top]=='['||(
            pop();
        }else{
            return 0;
```

ABCD+(

Output

```
Enter an expression:Not balanced
```

```
        }
    }
}
return (top==-1);
}
int main(){
    char expr[MAX];
    printf("Enter an expression:");
    scanf("%49s",expr);
    if(isBalanced(expr))
        printf("Balanced\n");
    else
        printf("Not balanced\n");
    return 0;
}

```

```c
#include <stdio.h>
#include <stdlib.h>
struct Node{
int data;
struct Node* next;
};
void printList(struct Node* head){
    struct Node* temp=head;
    while (temp!=NULL){
        printf("%d",temp->data);
        temp=temp->next;
    }
    printf("\n");
}
int main(){
    struct Node n3={30,NULL};
    struct Node n2={20,&n3};
    struct Node n1={10,&n2};
    printList(&n1);
    return 0;
}
```

stdin input (optional)

Output

```
102030
```

```c
#include <stdio.h>
#define MAX 5
int stack [MAX];
int top=-1;
void push(int value){
    if(top==MAX-1){
        printf("Overflow\n");
        return;
    }
    stack[++top]=value;
}
int pop(){
    if(top==-1){
        printf("Underflow\n");
        return -1;
    }
    return stack[top--];
}
int main()
{
push (10);
```

stdin input (optional)

Output

```
Popped:20
Popped:10
Underflow
Popped:-1
```

```
        printf("Underflow\n");
        return -1;
    }
    return stack[top--];
}
int main()
{
push (10);
push (20);
printf("Popped:%d\n",pop());
printf("Popped:%d\n",pop());
printf("Popped:%d\n",pop());
    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>
struct Node{
int data;
struct Node* next;
};
struct Node* reverseList(struct Node* head){
    struct Node* prev=NULL;
    struct Node* current=head;
    struct Node* next=NULL;
    while (current!=NULL){
        next=current->next;
        current->next=prev;
        prev=current;
        current=next;
    }
    return prev;
}
struct Node* insertEnd(struct Node* head,int value){
    struct Node* newNode=(struct Node*)malloc(sizeof(struct Node));
     newNode->data=value;
     newNode->next=NULL;
    if (head==NULL) return newNode;
    struct Node* temp=head;
    while (temp->next!=NULL)
        temp=temp->next;
    temp->next=newNode;
    return head;
}
void printlist(struct Node* head){
    struct Node* temp=head;
    while(temp!=NULL){
        printf("%d->",temp->data);
        temp=temp->next;
    }
    printf("NULL\n");
}
int main(){
    struct Node* head=NULL;
    head=insertEnd(head,10);
    head=insertEnd(head,20);
    head=insertEnd(head,30);
    head=insertEnd(head,40);
    printf("Original list:");
```

stdin input (optional)

Output

```
Original list:10->20->30->40->NULL
Reversed List:40->30->20->10->NULL
```

```c
#include <stdio.h>
#include <stdlib.h>
struct Node{
    int data;
    struct Node* next;
};
struct Node* insertAtHead(struct Node* head,int value){
    struct Node* newNode=malloc(sizeof(struct Node));
    newNode->data=value;
    newNode->next=head;
    head=newNode;
    return head;
}
void printlist(struct Node* head){
    while(head!=NULL){
        printf("%d->",head->data);
        head=head->next;
    }
    printf("NULL\n");
}
int main(){
    struct Node* head=NULL;
```

stdin input (optional)

Output

```
10->20->30->NULL
```

```c
}
void printlist(struct Node* head){
    while(head!=NULL){
        printf("%d->",head->data);
        head=head->next;
    }
    printf("NULL\n");
}
int main(){
    struct Node* head=NULL;
    head=insertAtHead(head,30);
    head=insertAtHead(head,20);
    head=insertAtHead(head,10);
    printlist(head);
    return 0;
}
```

```c
#include <stdio.h>
#define SIZE 5
int Q[SIZE], front=-1, rear=-1;
void enqueue(int val){
    if(rear==SIZE-1){printf("Overflow\n");return;}
    if (front==-1) front=0;
Q[++rear]=val;
}
int dequeue(){
    if(front==-1||front>rear){
        printf("Underflow\n");
        return -1;
    }
    int val=Q[front++];
    if(front>rear)front=rear=-1;
    return val;
}
int main(){
    enqueue(10);enqueue(20);enqueue(30);
    printf("%d\n",dequeue());
    printf("%d\n",dequeue());
    printf("%d\n",dequeue());
```

stdin input (optional)

Output

```
10
20
30
Underflow
-1
```

```
    int val=Q[front++];
    if(front>rear)front=rear=-1;
    return val;
}
int main(){
    enqueue(10);enqueue(20);enqueue(30);
    printf("%d\n",dequeue());
    printf("%d\n",dequeue());
    printf("%d\n",dequeue());
    printf("%d\n",dequeue());
    return 0;
}
```

Point out the error?

PEEK(stack):

return stack[top+1]

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```c
#include <stdio.h>
int stack[5],top=-1;
int peek(){
    if(top==-1) return -1;
    return stack[top];
}
int main(){
    stack[++top]=10;
    stack[++top]=20;
    printf("%d\n",peek());
    return 0;
}
```

Test Input

stdin input (optional)

Output

```
20
```

```c
#include <stdio.h>
#define MAX 5
int queue[MAX],front=0,rear=0,count=0;
void enqueue(int vehicle){
    if (count==MAX){
        printf("Queue full");
        return;
    }
    queue[rear]=vehicle;
    rear=(rear+1)%MAX;
    count++;
}
int main(){
    for(int i=1;i<=6;i++)enqueue(i);
    return 0;
}
```

stdin input (optional)

Output

```
Queue full
```

Point out the error.

if top == MAX:

Overflow

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```c
#include <stdio.h>
#define MAX 5
int stack[MAX],top=-1;
void push(int val){
    if(top==MAX-1){printf("Overflow\n");return;}
    stack[++top]=val;
}
int main(){
    for(int i=1;i<=6;i++) push(i);
    return 0;
}
```

Test Input

stdin input (optional)

Output

```
Overflow
```

Identify the issue in the below pseudocode.

POP():

if top == 0 → Underflow

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```c
#include <stdio.h>
#define MAX 5
int stack[MAX],top=-1;
int pop(){
    if(top==-1){printf("Underflow\n");return -1;}
    return stack[top--];
}
int main(){
    stack[++top]=10;
    printf("%d\n",pop());
    printf("%d\n",pop());
    return 0;
}
```

Test Input

stdin input (optional)

Output

```
10
Underflow
-1
```

Identify the issue in the below pseudocode.

POP():

if top == 0 → Underflow

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```c
#include <stdio.h>
#define MAX 5
int stack[MAX],top=-1;
int pop(){
    if(top==-1){printf("Underflow\n");return -1;}
    return stack[top--];
}
int main(){
    stack[++top]=10;
    printf("%d\n",pop());
    printf("%d\n",pop());
    return 0;
}
```

Test Input

stdin input (optional)

Output

```
10
Underflow
-1
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {int data; struct Node *prev,*next;}Node;
void deleteNode(Node** head,Node* del){
    if (del->prev) del->prev->next=del->next; else *head=del->next;
    if (del->next) del->next->prev= del->prev;
    free(del);
}
int main(){
    Node *a=malloc(sizeof(Node)),*b=malloc(sizeof(Node));
    *a=(Node){1,NULL,b};
    *b=(Node){2,a,NULL};
    Node* head=&a;
    deleteNode(&head,a);
    printf("%d\n",head->data);
}
```

Test Input

stdin input (optional)

Output

```
2
```

if (rear == MAX) return;

queue[rear++] = data;

}

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```c
#include <stdio.h>
#define MAX 5
int q[MAX],front=0,rear=0,cnt=0;
void enqueue(int d){
    if(cnt==MAX) return;
    q[rear]=d;
    rear=(rear+1)%MAX;
    cnt++;
}
int main(){
    enqueue(1);enqueue(2);
    printf("%d\n",cnt);
    return 0;
```

Test Input

stdin input (optional)

Output

```
2
```

deque[front] = data;

}

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```c
#include <stdio.h>
#define MAX 5
int dq[MAX], front=-1, rear=-1;
void insertFront(int d){
    if(front==0&rear==MAX-1)return;
    if(front==-1)front=rear=0;
    else if (front==0)front=MAX-1;
    else front--;
    dq[front]=d;
}
int main(){
insertFront(10);insertFront(20);
printf("%d\n",dq[front]);
    return 0;
```

Test Input

stdin input (optional)

Output

```
20
```

```c
#include <stdio.h>
#define MAX 5
int queue[MAX],front=-1,rear=-1;
void enqueue(int d){
    if((front==0&&rear==MAX-1)||(rear+1)%MAX==front)return;
    if(front==-1)front=0;
    rear=(rear+1)%MAX;
    queue[rear]=d;
}
int dequeue(){
    if(front==-1)return -1;
    int data=queue[front];
    if(front==rear)front=rear=-1;
    else front=(front+1)%MAX;
    return data;
}
int main(){
    enqueue(10);enqueue(20);
    printf("%d\n",dequeue());
    printf("%d\n",dequeue());
    printf("%d\n",dequeue());
    return 0;
}
```

stdin input (optional)

Output

```
10
20
-1
```

```c
#include <stdio.h>
#include <ctype.h>
int stack[50],top=-1;
void push(int v){stack[++top]=v;}
int pop(){return stack[top--];}
int evalPostfix(char* e){
    for (int i=0;e[i];i++){
        if(isdigit(e[i])) push(e[i]-'0');
        else{
            int op1=pop(),op2=pop();
            if(e[i]=='+')push (op2+op1);
            else if(e[i]=='-')push(op2-op1);
            else if(e[i]=='*')push(op2*op1);
            else push (op2/op1);
        }
    }
    return pop();
}
int main(){
    printf("%d\n",evalPostfix("62-"));
    return 0;
}
```

stdin input (optional)

Output

```
4
```